25. Ejercicio de Knowledge Priming

Jonathan Ricardo Serna Galina
20074902

Programación Asistida

Castillo Matemático: contexto del ejercicio 19 en la rama main. Revisión: 21 de octubre de 2026.

Borrador asistido por IA: debo revisarlo y redactar mi versión personal para cumplir la consigna sin IA.

1. Arquitectura

- Separo la interfaz en app.py y las reglas en game.py; conservo la partida en st.session_state.game como diccionario.

- Uso Exercise para los ejercicios y ExerciseGenerator para generarlos; cargo las reglas desde JSON.

- Mi jugador inicia con 100 de vida y 0 de oro; recorre 9 habitaciones. Un acierto da room * 10 monedas y permite avanzar.

- Un error resta 5 de vida; agotar el tiempo resta 10. Gano al completar las habitaciones y pierdo con vida menor o igual a cero.

2. Stack y versiones

- Declaro Python >=3.11; mi entorno registrado en .venv/pyvenv.cfg corresponde a Python 3.14.

- Mi uv.lock fija Streamlit 1.64.0 y streamlit-autorefresh 1.0.1; el proyecto está en 0.1.0.

- Gestiono dependencias con uv; uso json, random, time, dataclasses y pathlib de la biblioteca estándar.

3. Fuentes curadas

- Consulto primero app.py, game.py y config/; uso tests/test_game.py como referencia de los comportamientos comprobados.

- Reviso pyproject.toml y uv.lock para las dependencias, y docs/PLAN.md para las reglas y comandos del proyecto.

- Consulto docs/CLASS_DIAGRAM.md para las relaciones; GameState y GameRules describen diccionarios, no clases implementadas.

4. Estructura del proyecto

- En 19vo Ejercicio videojuego/: app.py presenta la interfaz; game.py genera ejercicios y actualiza la partida.

- En config/settings.json defino vida, daño y habitaciones; en config/exercises.json, dificultades, operaciones y tiempos.

- En tests/test_game.py mantengo comprobaciones de reglas e interfaz; en docs/ guardo el plan, diagramas e historial.

5. Convenciones de nombres

- Uso snake_case para funciones y variables (submit_answer, starting_gold) y PascalCase para clases (ExerciseGenerator).

- Mantengo identificadores, comentarios y docstrings en inglés; escribo los mensajes de la interfaz en español.

- Prefiero funciones pequeñas con una responsabilidad y nombres descriptivos; conservo claves existentes como n, x y z.

6. Ejemplo de código

Tomo este fragmento existente de game.py como referencia de sencillez; debo confirmar personalmente que refleja mi estilo.

```
def load_json(path: str | Path) -> dict:
    """Read UTF-8 JSON, relative to the current working directory."""
    with Path(path).open(encoding="utf-8") as file:
        return json.load(file)
```

Prefiero esta función directa: recibe una ruta, abre el archivo con with y devuelve el JSON sin crear clases innecesarias.

7. Antipatrones que evito

- No mezclo llamadas de Streamlit con las reglas de game.py, ni sustituyo el estado de sesión por variables globales compartidas.

- No duplico valores de configuración ni fijo la dificultad en el código; mantengo los niveles ordenados y cubriendo todas las habitaciones.

- No genero divisiones con resto ni restas negativas; tampoco bloqueo el temporizador con sleep.

- No añado clases, dependencias o funciones futuras sin necesidad; niebla, mascota y Wordle pertenecen a otras ramas, no a esta base.

- No ejecuto desde otra carpeta: mis rutas JSON son relativas al directorio de trabajo. Reviso este contexto si cambia el juego o en seis meses.